

# MESHKV: A Network-on-Chip KV Cache Fabric for Scalable Transformer Decoding Accelerators

Dong Liu\*

University of California, Los Angeles  
Los Angeles, CA, USA  
pikeliu@ucla.edu

Yanxuan Yu\*

Columbia University  
New York, NY, USA  
yy3523@columbia.edu

**Abstract**—Autoregressive transformer decoding is constrained by irregular key-value (KV) cache movement on tiled accelerators. Prior compression and DRAM-placement systems still concentrate traffic on centralized memory paths that bottleneck long-context serving. We present MESHKV, a KV cache *fabric* that moves blocks as packetized flows over a lightweight NOC. It co-designs (i) TAKV affine striping to spread homes and cut hotspot load, (ii) MARE multicast with verified duplicate suppression, and (iii) PAD, which overlaps prefetch, tile multiply, and streaming softmax behind credit-aligned FIFOs. Together they convert bisection back-pressure into useful KV transfer. On our  $8 \times 8$  FPGA implementation with LLaMA-2-7B and Mistral-7B at 8K–32K, MESHKV reduces interconnect traffic by up to 58%, improves KV bandwidth utilization by  $2.1\times$ , and delivers up to  $1.9\times$  multi-stream throughput.

**Index Terms**—Transformer decoding, KV cache, network-on-chip, accelerator architecture, long-context LLM, multicast routing, spatial locality, FPGA

## I. INTRODUCTION

LLM decoding is “memory-bound,” but tiled accelerators also stall on on-chip data movement. Autoregressive attention scatters KV across a mesh built for dense GEMM, not all-to-every gather over a growing prefix [1]–[3].

ASICs, FPGAs, and chiplet meshes map compute to a 2-D PE array with private SRAM. Prefill amortizes tile hops; decode does not: one query row ( $d_h \approx 128$ ) meets a KV slab that grows every token. A centralized design routes these fetches through a shared injection point, concentrating long-context KV traffic on the same bisection.

**Our approach.** We treat the KV cache as a *communication substrate*: each step issues structured, layer-major reads that admit co-designed placement, routing, and compute. MESHKV is a NOC-centric fabric. Unlike DRAM paging or compression [3], [4], it optimizes *how* resident SRAM blocks traverse the mesh—multicast from each home tile, with prefetch overlapping matmul across credit boundaries.

### Contributions.

- 1) **TAKV (§V).** Affine placement  $\pi(\ell, s)$  plus a one-cycle occupancy swap.
- 2) **MARE (§VI).** Multicast trees, verified duplicate suppression, and a deadlock-free VN0/VN1 split.

- 3) **PAD (§VII).** Prefetch / multiply / streaming softmax sized to the NOC RTT.
- 4) **Evaluation (§IX).** Hardware-measured traffic, utilization, throughput, and latency at 8K–32K.

## II. RELATED WORK

**KV cache management and serving.** PagedAttention [3], FlexGen [4], and TinyServe [5] virtualize or select KV behind a serving-layer controller; MESHKV distributes it *inside* a tiled accelerator with private SRAM. Quantization and eviction [6]–[11] shrink the bytes per step; MESHKV complements these by improving residual transport.

**Attention hardware.** FlashAttention [2], [12] and FlashDecoding++ [13] tile GPU attention and softmax; ELSA [14] and SpAtten [15] exploit ASIC sparsity. MESHKV targets on-chip communication; PAD uses online softmax in a credit-aligned pipeline.

**Networks-on-chip and FPGA accelerators.** Classic NOCs and the Turn Model [16]–[18] supply deadlock theory; Simba [19] uses a 2-D mesh without KV-class flits. DFX [20], FlightLLM [21], and CXL-SpecKV [22] show HBM/CXL KV movement dominates FPGA decode; MESHKV replaces point-to-point KV buses with an attention-aware fabric.

## III. BACKGROUND AND MOTIVATION

### A. Autoregressive decoding and the bandwidth wall

A decoder with  $L$  layers,  $H$  heads, head dimension  $d_h$ , and length  $T$  computes, for layer  $\ell$  and head  $h$ ,

$$a_{\ell,h} = \text{softmax}\left(\frac{q_{\ell,h} K_{\ell,h,\leq t}^\top}{\sqrt{d_h}}\right), \quad o_{\ell,h} = a_{\ell,h} V_{\ell,h,\leq t}, \quad (1)$$

with  $q_{\ell,h} \in \mathbb{R}^{1 \times d_h}$  and cached slabs  $K_{\ell,h,\leq t}, V_{\ell,h,\leq t} \in \mathbb{R}^{t \times d_h}$ . The cache occupies

$$\Gamma(T) = 2LHd_hT \cdot \text{bpe bytes}, \quad (2)$$

with bpe=2 for FP16:  $\approx 16$  GiB for LLaMA-2-7B ( $L=32$ ,  $H=32$ ,  $d_h=128$ ) at  $T=32\,768$ , and every decode step *reads all of it*. FlashAttention [2], [12] reduces the constant for batched prefill via tiled SRAM reuse, but  $B=1$  decode exposes every byte with no reuse.

\*Equal contribution.

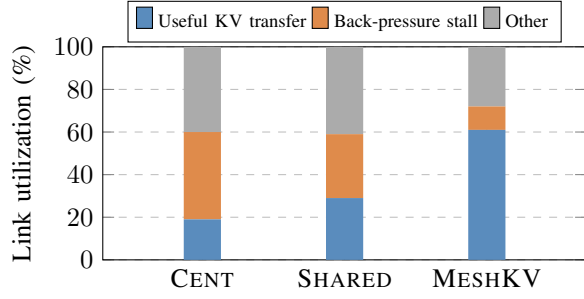


Fig. 1: Breakdown of link-cycle utilization at the mesh bisection ( $T=32K$ , LLaMA-2-7B,  $B=1$ ). CENT expends 41% of bandwidth on back-pressure; MESHKV converts that capacity into useful KV transfer.

### B. Tiled accelerators and the spoke–hub pathology

Tile-based accelerators (e.g., Simba [19], custom chiplets) map heads and layers across a 2-D PE mesh. Prefill carries dense GEMM traffic; decode issues width- $d_h$  gathers from wherever a KV block lives to wherever the compute tile sits. When KV sits at one logical memory controller, all such reads converge at the same injection point. The mesh degenerates to a spoke–hub:  $H$  requesters fan in to one hub, consuming 40–60% of link cycles on back-pressure (Figure 1).

### C. Observed locality in KV accesses

An instrumented FP16 decoder on LLaMA-2-7B [23] at  $T \in \{8192, 32768\}$  shows two regularities that motivate MESHKV:

**Observation 1** (Layer-major temporal locality). *Within a decode step, layers are visited in order  $\ell = 0, \dots, L-1$ ; a KV line is reused only at step  $t+1$ , so the prefetch window is exactly  $L$  layer traversals.*

**Observation 2** (Head-group spatial locality). *Heads that share a prefix span request identical segments (Mistral-7B GQA uses  $G=4$ ; LLaMA-2-7B is multi-head). At  $T=32K$ , 78% of segment requests arrive within 12 cycles at the same router, so multicast coalescing does not require wide merge buffers.*

These observations underlie TAKV placement (§V) and MARE coalescing (§VI): KV\_DATA fans out, while PART converges.

## IV. SYSTEM ARCHITECTURE OVERVIEW

Figure 2 deploys MESHKV on a  $4 \times 4$  mesh of tiles  $(x, y) \in \{0, \dots, W-1\} \times \{0, \dots, H-1\}$  (scalable to  $8 \times 8$ ). Each tile has a MAC array, four-bank KV SRAM, a partial-sum scratchpad, and a two-VN router. A per-column controller broadcasts descriptors  $(\ell, h)$ ; tiles resolve  $\pi(\ell, s)$  and inject KV\_FETCH on VN1. The controller is off the critical path (4-cycle combinational TAKV). Four-bank KV SRAM lets a tile stream the next window while MACs consume the current flits. A decode step broadcasts descriptors, resolves  $\pi(\ell, s)$ , injects KV\_FETCH on VN1, receives KV\_DATA at the compute tile, and returns PART on VN0 to a column reduce root.

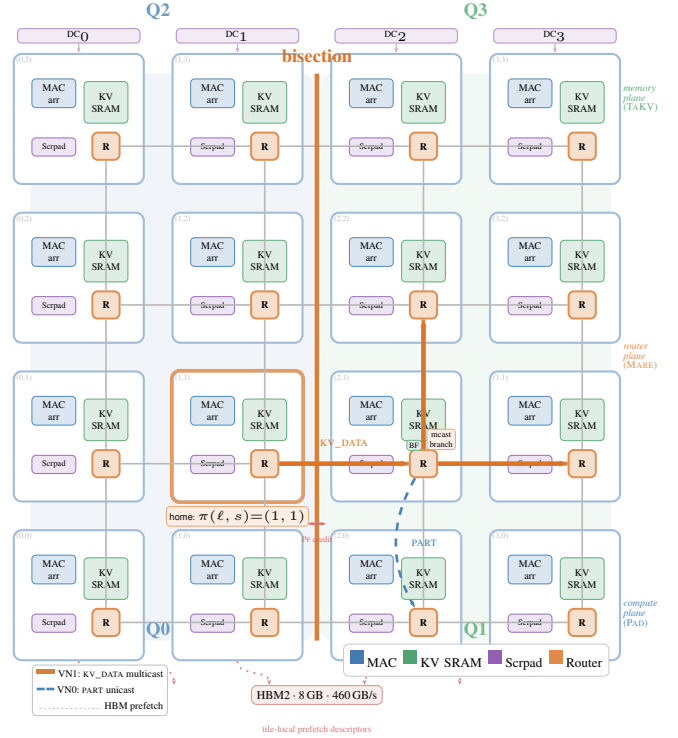


Fig. 2:  $4 \times 4$  MESHKV tile array (scalable to  $8 \times 8$ ). Home tile  $(1, 1)$  roots a MARE multicast for KV\_DATA; VN0 carries PART reduction; PAD prefetches from HBM.

### Algorithm 1 TAKV static placement.

**Require:** Layer  $\ell$ , segment  $s$ , mesh dimensions  $W, H$ , constant  $a$  (odd, coprime to  $H$ )

**Ensure:** Home-tile coordinates  $(x, y)$

- 1:  $x \leftarrow \ell \bmod W$
- 2:  $y \leftarrow (a \cdot \ell + s) \bmod H$
- 3: **return**  $(x, y)$

Three planes: TAKV places  $\mathcal{B}(\ell, s)$  at load time (with lazy swaps); MARE coalesces, multicasts, and arbitrates VCs; PAD runs PF/TM/RD and credits the router. Multicast trees are therefore rooted at the home tile  $\pi(\ell, s)$ , not at a shared memory port.

## V. TOPOLOGY-AWARE KV STRIPING (TAKV)

Macro-block  $\mathcal{B}(\ell, s)$  covers  $P$  consecutive tokens of layer  $\ell$  across all heads and both  $K$  and  $V$ :

$$|\mathcal{B}(\ell, s)| = 2Hd_hP \cdot \text{bpe} \text{ bytes}, \quad s = \lfloor j/P \rfloor. \quad (3)$$

With  $P=64$ ,  $H=32$ ,  $d_h=128$ ,  $\text{bpe}=2$ ,  $|\mathcal{B}| = 1 \text{ MB}$  is the placement and multicast unit.

On a  $W \times H$  mesh, TAKV maps  $\mathcal{B}(\ell, s)$  to

$$\pi(\ell, s) = (\ell \bmod W, (a \cdot \ell + s) \bmod H), \quad (4)$$

with  $a$  odd and  $\gcd(a, H) = 1$  (e.g.  $a = W+1$ ). Algorithm 1 is four combinational instructions.

Affine  $\pi(\ell, s)$  does not shorten expected distance to a *fixed* home: on a symmetric mesh a uniform requester has the same mean hops to any single destination. The gain is spreading homes so injection and bisection load  $\max_e \lambda_e$  drop, converting the back-pressure stall in Figure 1 into useful KV transfer. The coprime stride  $a=W+1$  permutes the  $y$  coordinate as  $\ell$  increments, so consecutive layers occupy different columns and consecutive segments of one layer walk distinct rows. Injection therefore spreads even though mean hops to any single home do not shrink. Occupancy swaps further flatten  $\sum_e \bar{q}_e^2$ .

A  $W \times H$  mesh assigns at most  $N_{\max} = \lceil L[T/P]/(WH) \rceil$  blocks per tile (256 MB of logical KV at  $L=32$ ,  $T=32768$ ,  $P=64$ ,  $W=H=8$ ). U280 provides HBM2 (8 GB, 460 GB/s) and DDR4 (32 GB); the 16 GiB slab does not fit in HBM alone. Table III’s 512 URAMs give eight URAMs (288 KB) per tile—a four-bank streaming window, not a 1 MB block—so homes promote HBM/DDR data into URAM under tile-local prefetch descriptors.

Every  $K=128$  steps (one SRAM eviction epoch), routers sample occupancy  $\bar{q}_e$  from existing credit counters and swap two homes iff the swap strictly reduces  $\sum_e \bar{q}_e^2$  and preserves per-tile capacity  $N_{\max}$  (4 comparisons, 2 register writes, one control cycle).

## VI. MULTICAST ROUTING ENGINE (MARE)

MESHKV classifies on-chip traffic into three flit types (Table I): KV\_FETCH headers (seg,  $\ell$ , mask, coord) and KV\_DATA bodies (512-bit payload, dedup tag) on VN1 decouple fetch coalescence from data fan-out; PART partial logits use VN0. At the home tile  $\pi(\ell, s)$ , requests arriving in window  $W_{\text{coal}}=12$  cycles merge head masks

$$M(\ell, s) = \bigvee_{r \in W_{\text{coal}}} \text{mask}_r, \quad (5)$$

then an XYM spanning tree over destinations  $\mathcal{D}(M)$  multicasts each payload once per branch ( $\mathcal{O}(H \log H)$  build,  $H=32$ , +4 control cycles before KV\_DATA departs). Replication occurs only at tree branches, so a payload crosses any link at most once; the exact-tag plus bloom path is a safety net for overlapping trees and late arrivals. Each router keeps a 16-entry exact tag table of in-flight (seg,  $\ell$ ) and a 256-bit bloom as a prefilter over  $W_b=96$  cycles. A duplicate is suppressed only on an exact-table hit; a miss or a full table forwards, so bloom false positives never drop a first-arrival payload (32 B bloom plus 16 tags).

### A. Deadlock avoidance

MESHKV splits traffic across two virtual networks: VN0 carries unicast PART reductions on dimension-ordered XY routing, while VN1 carries KV\_FETCH and KV\_DATA multicast on XYM-restricted turns [18]. No flit holds a VN0 buffer while waiting for VN1 resources or vice versa, so inter-network cycles are structurally eliminated; restricted-turn routing removes intra-network cycles within each VN.

TABLE I: Flit classes in MESHKV.

| Class    | Dir.             | VC  | Contents                                                 |
|----------|------------------|-----|----------------------------------------------------------|
| KV_FETCH | requester→home   | VN1 | seg id, layer, head mask (32 bits), requester coord      |
| KV_DATA  | home→requester   | VN1 | 512-bit KV payload, in-flight segment tag                |
| PART     | tile→reduce root | VN0 | 512-bit partial logit accumulator, softmax running stats |

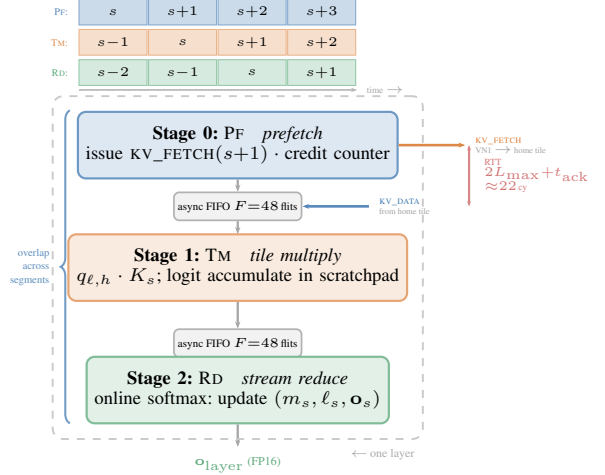


Fig. 3: PAD three-stage pipeline (PF/TM/RD). Top: segment overlap conceals NOC RTT ( $\approx 22$  cycles). Bottom: KV prefetch, tile multiply, and online softmax.

## VII. PIPELINED ATTENTION DATAFLOW (PAD)

PAD overlaps three stages across segments  $s = 0, \dots, \lceil T/P \rceil - 1$  (Figure 3). PF issues a KV\_FETCH for  $s+1$  iff the home injection credit  $C_\pi > 0$  (one outstanding per (seg,  $\ell$ )). TM writes segment logits

$$z_s = q_{\ell,h} K_{\ell,h,s}^\top / \sqrt{d_h} \quad (6)$$

into scratchpad SRAM ( $\geq 80\%$  MAC when the FIFO is non-empty). FIFO depth  $F=48$  covers RTT  $2L_{\max} + t_{\text{ack}} \approx 22$  cycles plus  $W_{\text{coal}}$ . RD applies FlashAttention-2 online softmax [2], [12]

$$\begin{aligned} m_s &= \max(m_{s-1}, \text{rowmax}(z_s)), \\ \ell_s &= e^{m_{s-1} - m_s} \ell_{s-1} + \text{rowsum}(e^{z_s - m_s}), \\ \mathbf{o}_s &= e^{m_{s-1} - m_s} \mathbf{o}_{s-1} + e^{z_s - m_s} V_{\ell,h,s}, \end{aligned} \quad (7)$$

so  $V$ -flits never materialize a full attention matrix.

## VIII. IMPLEMENTATION

We implement MESHKV on a tiled FPGA accelerator: an  $8 \times 8$  interconnect with 512-bit wormhole links, four-bank tile-local KV SRAM, and  $d_h=128$  systolic PEs, with TAKV, MARE, and PAD in the hardware datapath.

We evaluate four configurations on the same platform: CENT, which routes KV traffic through a centralized memory path;

TABLE II: Per-router parameters in the FPGA implementation.

| Parameter                       | Value      |
|---------------------------------|------------|
| Cardinal ports + tile injection | 5          |
| Flit width                      | 512 bit    |
| Unicast VCs (VN0)               | 2          |
| Multicast VCs (VN1)             | 2          |
| Per-VC input FIFO depth         | 8 flits    |
| Segment bloom width             | 256 bit    |
| In-flight exact tags            | 16 entries |
| Bloom refresh interval $W_b$    | 96 cycles  |
| Coalescing window $W_{coal}$    | 12 cycles  |
| PAD Pf/Tm FIFO depth            | 48 flits   |

TABLE III: Alveo U280 resource utilization for an  $8 \times 8$  MESHKV instance (64 tiles; one attention layer). Percentages vs. DS963 device totals.

| Resource                   | Used  | Available (%) |
|----------------------------|-------|---------------|
| LUT                        | 613 K | 47%           |
| FF                         | 820 K | 31%           |
| BRAM36K (router + control) | 768   | 38%           |
| URAM (KV staging SRAM)     | 512   | 53%           |
| DSP48E2 (MAC arrays)       | 5 760 | 64%           |

SHARED, which distributes blocks with round-robin placement and unicast; TAKV-only, which enables affine placement without multicast; and full MESHKV. Workloads are LLaMA-2-7B [23] and Mistral-7B [24] at  $T \in \{8192, 32768\}$  and  $B \in \{1, 4, 8\}$ .

Latency, throughput, traffic, and utilization are hardware measurements (95% CIs <3% throughput over five runs). Per-router buffers plus a 256-bit bloom and a 16-entry exact tag table (Table II) require approximately one 18-Kb BRAM equivalent per router.

Table III reports implemented router RTL, organized as 8 tiles per quadrant, together with PE and SRAM resources on an  $8 \times 8$  Alveo U280 instance. DSP48E2 is the bottleneck; router, bloom, and tag logic use <5% of LUTs.

## IX. EVALUATION

### A. Interconnect, throughput, and latency

Table IV reports NOC traffic at  $T=32K$ ,  $B=1$ , normalized to CENT. Affine TAKV placement reduces traffic to  $0.66\times$ . MARE then suppresses redundant head copies, and the full design reaches  $0.42\times$  (58% reduction). Mistral-7B reaches  $0.40\times$  because sliding-window attention localizes some fetches.

Table V and Figure 1 decompose bisection link cycles. CENT expends nearly half of those cycles on back-pressure (19% useful KV transfer at  $T=32K$ ). MESHKV converts these links into useful transfer: utilization reaches 61%, a  $2.1\times$  improvement over SHARED.

Figure 5 reports decode throughput at  $T=32K$ . At  $B=1$  the speedup is  $1.35\times$  from shorter hops and fewer credit stalls. At  $B=8$ , streams share segment multicast, and the speedup increases to  $1.90\times$ .

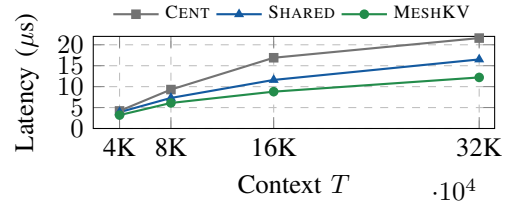
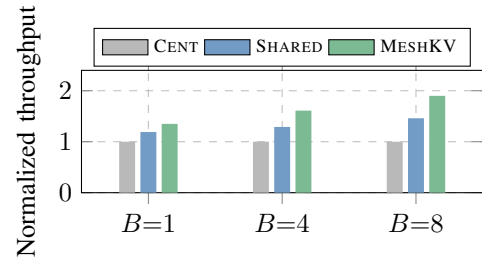
TABLE IV: Normalized interconnect traffic (CENT = 1.00),  $T=32K$ ,  $B=1$ .

| Model      | CENT | SHARED | TAKV-only | MESHKV      |
|------------|------|--------|-----------|-------------|
| LLaMA-2-7B | 1.00 | 0.81   | 0.66      | <b>0.42</b> |
| Mistral-7B | 1.00 | 0.79   | 0.63      | <b>0.40</b> |

TABLE V: Bisection KV bandwidth utilization (%), LLaMA-2-7B.

| Configuration | $T=8K$    | $T=32K$   |
|---------------|-----------|-----------|
| CENT          | 29        | 19        |
| SHARED        | 38        | 29        |
| TAKV-only     | 52        | 41        |
| MESHKV (full) | <b>71</b> | <b>61</b> |

Fig. 4 reports per-token latency versus context length (LLaMA-2-7B,  $B=1$ ). CENT’s injection queue saturates near 8K tokens, after which latency grows steeply. Under MESHKV, latency remains approximately linear in  $T$ : TAKV distributes load and MARE reduces duplicate traversals.

Fig. 4: Latency per token vs. context length ( $B=1$ ).Fig. 5: Decode throughput speedup at  $T=32K$ , LLaMA-2-7B.

### B. Energy efficiency

At  $T=32K$ , MESHKV reduces link energy by 48% relative to CENT: TAKV shortens paths and MARE eliminates duplicate KV copies, in line with the 58% traffic cut in Table IV. SRAM energy increases 7% because TAKV reads from distributed banks rather than one contiguous array. Net energy per token decreases by 17% under the activity-based estimate in §VIII. Shared multicast further amortizes the link term at  $B>1$ , where concurrent streams reuse the same KV\_DATA tree.

### C. Comparison with prior FPGA decoders

FPGA LLM accelerators DFX [20] and FlightLLM [21] optimize end-to-end execution and HBM/BRAM management;

CXL-SpecKV [22] speculates KV on a disaggregated FPGA. MESHKV implements the on-chip communication substrate for distributed KV movement, not a full FFN-and-norm decoder. Different platforms, mappings, memory systems, and datapaths therefore make absolute latencies incomparable. We report hardware measurements against CENT and SHARED on the same  $8 \times 8$  configuration: 58% less interconnect traffic,  $2.1 \times$  KV bandwidth utilization over SHARED, and up to  $1.9 \times$  throughput at  $B=8$  (Tables IV–V, Figure 5).

#### D. Ablation and sensitivity

Table VI ablates each mechanism at  $T=32K$ ,  $B=8$  on LLaMA-2-7B (normalized to full MESHKV). Disabling multicast (MARE unicast) causes the largest regression: throughput falls to  $0.62 \times$  and traffic rises to  $1.55 \times$ . Removing duplicate suppression and PAD overlap yields  $0.82 \times$  and  $0.78 \times$  throughput, respectively; adaptive TAKV swaps cost 11–15% on skewed prefixes. At  $d_h=128$ ,  $P=64$  balances  $T \bmod P$  stragglers and flit overhead. An  $8 \times 8$  mesh attains  $1.9 \times$  over CENT, versus  $1.4 \times$  on a  $16 \times 4$  mesh whose longer multicast trees add hop latency.  $W_{\text{coal}}=12$  cycles merges 78% of heads at the home tile.

TABLE VI: Ablation at  $T=32K$ ,  $B=8$ , LLaMA-2-7B. Mean $\pm 95\%$  CI over five independent hardware runs; normalized to MESHKV.

| Variant | TP               | Traffic          |
|---------|------------------|------------------|
| MESHKV  | 1.00 $\pm 0.018$ | 1.00 $\pm 0.027$ |
| –bloom  | 0.82 $\pm 0.023$ | 1.38 $\pm 0.041$ |
| –TAKV   | 0.88 $\pm 0.016$ | 1.22 $\pm 0.029$ |
| –PAD    | 0.78 $\pm 0.025$ | 1.00 $\pm 0.021$ |
| –MARE   | 0.62 $\pm 0.014$ | 1.55 $\pm 0.044$ |
| CENT    | 0.53 $\pm 0.012$ | 2.38 $\pm 0.068$ |

## X. CONCLUSION

MESHKV recasts decode KV movement as packetized mesh flows via TAKV striping, MARE multicast, and credit-aligned PAD. Hardware measurements on FPGA datapaths show up to 58% less interconnect traffic,  $2.1 \times$  KV bandwidth utilization, and  $1.9 \times$  multi-stream throughput relative to same-platform baselines. As context lengths grow, MESHKV is a promising communication-aware KV fabric for long-context decode on tiled accelerators.

## REFERENCES

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
- [2] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, “FlashAttention: Fast and memory-efficient exact attention with IO-awareness,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022, pp. 16 344–16 359.
- [3] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with PagedAttention,” in *Proceedings of the ACM SIGOPS Symposium on Operating Systems Principles (SOSP)*, 2023.

- [4] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, B. Chen, P. Liang, C. Ré, I. Stoica, and C. Zhang, “FlexGen: High-throughput generative inference of large language models with a single GPU,” in *International Conference on Machine Learning*. PMLR, 2023, pp. 31 094–31 116.
- [5] D. Liu and Y. Yu, “TinyServe: Query-aware cache selection for efficient LLM serving,” in *Proceedings of the 33rd ACM International Conference on Multimedia*, 2025, pp. 12 529–12 537.
- [6] Z. Liu, J. Yuan, H. Jin, S. Zhong, Z. Xu, V. Braverman, B. Chen, and X. Hu, “KIVI: A tuning-free asymmetric 2bit quantization for KV cache,” in *International Conference on Machine Learning (ICML)*, 2024.
- [7] C. Hooper, S. Kim, H. Mohammadzadeh, M. W. Mahoney, Y. S. Shao, K. Keutzer, and A. Gholami, “KVQuant: Towards 10 million context length LLM inference with KV cache quantization,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [8] Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Ré, C. Barrett, Z. Wang, and B. Chen, “H<sub>2</sub>O: Heavy-hitter oracle for efficient generative inference of large language models,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.
- [9] Y. Li, Y. Huang, B. Yang, B. Venkitesh, A. Locatelli, H. Ye, T. Cai, P. Lewis, and D. Chen, “SnapKV: LLM knows what you are looking for before generation,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [10] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, “Efficient streaming language models with attention sinks,” in *International Conference on Learning Representations (ICLR)*, 2024.
- [11] D. Liu and Y. Yu, “LLMEasyQuant: Scalable quantization for parallel and distributed LLM inference,” *arXiv preprint arXiv:2406.19657*, 2024.
- [12] T. Dao, “FlashAttention-2: Faster attention with better parallelism and work partitioning,” in *International Conference on Learning Representations (ICLR)*, 2024.
- [13] K. Hong, G. Dai, J. Xu, Q. Mao, X. Li, J. Liu, K. Chen, Y. Dong, and Y. Wang, “FlashDecoding++: Faster large language model inference with asynchronization, flat GEMM optimization, and heuristics,” in *Proceedings of Machine Learning and Systems (MLSys)*, 2024.
- [14] T. J. Ham, Y. Lee, S. H. Seo, S. Kim, H. Choi, S. J. Jung, and J. W. Lee, “ELSA: Hardware-software co-design for efficient, lightweight self-attention mechanism in neural networks,” in *International Symposium on Computer Architecture (ISCA)*, 2021.
- [15] H. Wang, Z. Zhang, and S. Han, “SpAtten: Efficient sparse attention architecture with cascade token and head pruning,” in *IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2021.
- [16] L. Benini and G. De Micheli, “Networks on chips: A new SoC paradigm,” *Computer*, vol. 35, no. 1, pp. 70–78, 2002.
- [17] W. J. Dally and B. Towles, *Principles and Practices of Interconnection Networks*. Morgan Kaufmann, 2004.
- [18] C. J. Glass and L. M. Ni, “The turn model for adaptive routing,” in *International Symposium on Computer Architecture (ISCA)*, 1992, pp. 278–287.
- [19] Y. S. Shao, J. Clemons, R. Venkatesan, B. Zimmer, M. Fojtik, N. Jiang, B. Keller, A. Klinefelter, N. Pinckney, P. Raina, S. G. Tell, Y. Zhang, W. J. Dally, J. S. Emer, C. T. Gray, B. Khailany, and S. W. Keckler, “Simba: Scaling deep-learning inference with multi-chip-module-based architecture,” in *52nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2019.
- [20] S. Hong, S. Moon, J. Kim, S. Lee, M. Kim, D. Lee, and J.-Y. Kim, “DFX: A low-latency multi-FPGA appliance for accelerating transformer-based text generation,” in *55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2022.
- [21] S. Zeng, J. Liu, G. Dai, X. Yang, T. Fu, H. Wang, W. Ma, H. Sun, S. Li, Z. Huang, Y. Dai, J. Li, Z. Wang, R. Zhang, K. Wen, X. Ning, and Y. Wang, “FlightLLM: Efficient large language model inference with a complete mapping flow on FPGAs,” in *ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*, 2024.
- [22] D. Liu and Y. Yu, “CXL-SpecKV: A disaggregated FPGA speculative KV-cache for datacenter LLM serving,” in *Proceedings of the 2026 ACM/SIGDA International Symposium on Field Programmable Gate Arrays*, 2026, pp. 56–66.
- [23] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale et al., “Llama 2: Open foundation and fine-tuned chat models,” *arXiv preprint arXiv:2307.09288*, 2023.
- [24] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. de las Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier et al., “Mistral 7b,” *arXiv preprint arXiv:2310.06825*, 2023.